TP : Mise en place d'une architecture microservices avec Spring Boot

1. Contexte général

L'objectif de ce TP est de concevoir et de mettre en œuvre une architecture microservices avec Spring Boot et Spring Cloud, permettant de gérer un catalogue de produits et les commandes des clients.

L'application sera composée de deux microservices métiers :

- ms-produits : gestion du catalogue des produits.
- ms-commandes : gestion des commandes et de leurs lignes.

Elle intégrera également cinq composants d'infrastructure :

- Eureka Server : découverte et enregistrement des microservices.
- OpenFeign : communication entre les microservices métiers.
- API Gateway : point d'entrée unique de l'application.
- Spring Cloud Config Server : centralisation de la configuration.
- Keycloak : authentification et gestion des rôles et des autorisations.

Architecture cible



Principe important : OpenFeign est une bibliothèque intégrée à ms-commandes, et non un serveur indépendant. Eureka et Config Server sont, eux, des applications distinctes. Keycloak est également un serveur externe à l'application métier.

2. Objectifs pédagogiques

À l'issue du TP, l'étudiant devra être capable de :

1. Créer des microservices avec Spring Boot.
2. Concevoir des API REST indépendantes.

3. Enregistrer et découvrir les services avec Eureka.
4. Configurer les appels interservices avec OpenFeign.
5. Centraliser la configuration avec Spring Cloud Config.
6. Mettre en place une passerelle avec Spring Cloud Gateway.
7. Sécuriser les API avec Keycloak et OAuth 2.0 / OpenID Connect.
8. Persister les données dans des bases de données distinctes.
9. Tester les échanges entre services et les mécanismes de sécurité.

3. Travail demandé et fonctionnalités métier

3.1. Microservice ms-produits

Ce microservice permet de gérer le catalogue des produits.

Fonctionnalités attendues :

- Ajouter un produit.
- Consulter un produit par son identifiant.
- Lister les produits disponibles.
- Rechercher un produit par son nom.
- Modifier les informations d'un produit.
- Supprimer un produit.
- Activer ou désactiver un produit.
- Vérifier la disponibilité d'un produit avant une commande.

Un produit possède au minimum un identifiant, une référence, un nom, une description, un prix, une quantité en stock et un état d'activation.

3.2. Microservice ms-commandes

Ce microservice gère les commandes des clients.

Fonctionnalités attendues :

- Créer une commande comportant un ou plusieurs produits.
- Consulter une commande par son identifiant.
- Lister les commandes.
- Consulter les commandes d'un client.
- Ajouter des lignes à une commande, selon les règles métier retenues.
- Calculer le montant total d'une commande.
- Modifier le statut d'une commande.
- Vérifier l'existence et la disponibilité des produits en interrogeant ms-produits via OpenFeign.

Statuts possibles : EN_ATTENTE, VALIDEE, EXPEDIEE, LIVREE et ANNULEE.

Règle de conception : chaque microservice possède sa propre base de données. Le microservice des commandes ne doit pas accéder directement aux tables du microservice des produits.

4. Diagrammes de classes UML des microservices métiers

4.1. Diagramme de classes de ms-produits

```
classDiagram
    class Produit {
```

```
        +Long id
        +String reference
        +String nom
        +String description
        +BigDecimal prix
        +Integer quantiteStock
        +Boolean actif
    }

    class ProduitController {
        +creerProduit()
        +obtenirProduit(id)
        +listerProduits()
        +rechercherProduits(nom)
        +modifierProduit(id)
        +supprimerProduit(id)
        +verifierDisponibilite(id, quantite)
    }

    class ProduitService {
        +creer(demande)
        +obtenirParId(id)
        +lister()
        +rechercher(nom)
        +modifier(id, demande)
        +supprimer(id)
        +estDisponible(id, quantite)
    }

    class ProduitRepository {
        +findByReference(reference)
        +findByNomContainingIgnoreCase(nom)
        +findByActifTrue()
    }

    class ProduitDTO {
        +Long id
        +String reference
        +String nom
        +BigDecimal prix
        +Integer quantiteStock
        +Boolean actif
    }

    ProduitController --> ProduitService
    ProduitService --> ProduitRepository
    ProduitService ..> ProduitDTO
    ProduitRepository --> Produit : persiste
```

Organisation suggérée des packages :

```
ms-produits/
└── src/main/java/com/formation/produits/
    ├── controller/
    │   └── ProduitController.java
    ├── service/
    │   └── ProduitService.java
    ├── repository/
    │   └── ProduitRepository.java
    ├── entity/
    │   └── Produit.java
    ├── dto/
    │   └── ProduitDTO.java
```

```
    ├── exception/
    └── config/
```

4.2. Diagramme de classes de ms-commandes

Le microservice des commandes conserve les informations utiles au moment de la commande, notamment le prix unitaire appliqué. Il ne conserve pas une relation JPA directe vers l'entité Produit de l'autre microservice : les deux applications sont indépendantes.

```
classDiagram
    class Commande {
        +Long id
        +String numero
        +String clientId
        +LocalDateTime dateCommande
        +StatutCommande statut
        +BigDecimal montantTotal
        +calculerMontantTotal()
        +changerStatut(statut)
    }

    class LigneCommande {
        +Long id
        +Long produitId
        +String referenceProduit
        +String nomProduit
        +Integer quantite
        +BigDecimal prixUnitaire
        +calculerSousTotal()
    }

    class StatutCommande {
        <<enumeration>>
        EN_ATTENTE
        VALIDEE
        EXPEDIEE
        LIVREE
        ANNULEE
    }

    class CommandeController {
        +creerCommande(demande)
        +obtenirCommande(id)
        +listerCommandes()
        +listerParClient(clientId)
        +changerStatut(id, statut)
    }

    class CommandeService {
        +creer(demande)
        +obtenirParId(id)
        +listerParClient(clientId)
        +validerDisponibilites(demande)
        +calculerTotal(lignes)
        +changerStatut(id, statut)
    }

    class CommandeRepository {
        +findByNumero(numero)
        +findByClientId(clientId)
    }
```

```
    class ProduitClient {
        <<FeignClient>>
        +obtenirProduit(id)
        +verifierDisponibilite(id, quantite)
    }

    class CommandeDTO {
        +Long id
        +String numero
        +String clientId
        +List~LigneCommandeDTO~ lignes
        +BigDecimal montantTotal
        +String statut
    }

    Commande "1" *-- "1..*" LigneCommande : contient
    Commande --> StatutCommande
    CommandeController --> CommandeService
    CommandeService --> CommandeRepository
    CommandeService --> ProduitClient : appelle
    CommandeService ..> CommandeDTO
    CommandeRepository --> Commande : persiste
```

Organisation suggérée des packages :

```
ms-commandes/
└── src/main/java/com/formation/commandes/
    ├── controller/
    │   └── CommandeController.java
    ├── service/
    │   └── CommandeService.java
    ├── repository/
    │   └── CommandeRepository.java
    ├── entity/
    │   ├── Commande.java
    │   ├── LigneCommande.java
    │   └── StatutCommande.java
    ├── dto/
    ├── client/
    │   └── ProduitClient.java
    ├── exception/
    └── config/
```

4.3. Règles de gestion à implémenter

10. Une commande contient au moins une ligne.
11. La quantité commandée doit être strictement positive.
12. Le prix unitaire est celui obtenu auprès de ms-produits au moment de la commande.
13. Le produit doit exister et être actif.
14. Le montant total est la somme des sous-totaux des lignes.
15. Une commande livrée ne peut plus être annulée directement.
16. Si ms-produits est indisponible, la création de commande doit échouer proprement, avec une réponse explicite.
17. Pour cette première version, le stock est vérifié mais sa réservation et sa décrémentation sont laissées à une extension du TP.

5. Diagrammes des composants d’infrastructure

Les composants d’infrastructure ne sont pas tous des microservices métiers. Pour eux, les diagrammes ci-dessous représentent principalement les classes de configuration et les interactions à mettre en place.

5.1. Eureka Server — découverte des services

```
classDiagram
    class EurekaServerApplication {
        +main(args)
    }
    class EurekaServerConfig {
        +activerEurekaServer()
    }
    class ServiceRegistry {
        <<registre logique>>
        +enregistrer(instance)
        +rechercher(serviceId)
        +retirer(instanceId)
    }
    class ServiceInstance {
        +String serviceId
        +String host
        +Integer port
        +String status
    }
    EurekaServerApplication --> EurekaServerConfig
    EurekaServerConfig --> ServiceRegistry
    ServiceRegistry o-- ServiceInstance
```

Travail demandé :

- Créer le projet eureka-server.
- Ajouter la dépendance Eureka Server.
- Activer le serveur avec @EnableEurekaServer.
- Configurer le tableau de bord Eureka.
- Enregistrer ms-produits, ms-commandes, api-gateway et config-server lorsque cela est nécessaire.

5.2. OpenFeign — communication interservices

OpenFeign est une bibliothèque utilisée dans ms-commandes. Il ne nécessite donc pas de projet serveur supplémentaire.

```
classDiagram
    class CommandeService {
        +creer(demande)
        +validerDisponibilites(demande)
    }
    class ProduitClient {
        <<FeignClient>>
        +obtenirProduit(id)
        +verifierDisponibilite(id, quantite)
    }
    class FeignConfiguration {
        +configurerTimeouts()
        +configurerGestionErreurs()
    }
    class ProduitsAPI {
        <<API REST distante>>
        +GET /api/produits/id
        +GET /api/produits/id/disponibilite
    }
    CommandeService --> ProduitClient
    ProduitClient --> ProduitsAPI : HTTP
    ProduitClient ..> FeignConfiguration
```

Travail demandé :

- Ajouter Spring Cloud OpenFeign à ms-commandes.
- Activer les clients Feign avec @EnableFeignClients.
- Déclarer une interface ProduitClient annotée avec @FeignClient(name = "ms-produits").
- Faire utiliser à Feign le nom du service enregistré dans Eureka.
- Configurer des délais d'attente et la gestion des erreurs.

L'étudiant devra démontrer qu'une commande peut récupérer les informations d'un produit sans connaître directement l'adresse IP de ms-produits.

5.3. API Gateway — point d'entrée unique

```
classDiagram
    class GatewayApplication {
        +main(args)
    }
    class GatewayRoutesConfig {
        +routes()
    }
    class SecurityConfig {
        +securityWebFilterChain()
    }
    class JwtAuthenticationFilter {
        +validerJeton(jwt)
    }
    class EurekaDiscovery {
        <<service discovery>>
        +resoudreService(serviceId)
    }
    GatewayApplication --> GatewayRoutesConfig
    GatewayApplication --> SecurityConfig
    SecurityConfig --> JwtAuthenticationFilter
    GatewayRoutesConfig --> EurekaDiscovery
```

Travail demandé :

- Créer api-gateway.
- Déclarer les routes vers les deux services métiers.
- Utiliser la découverte de services pour le routage.
- Protéger les routes avec la validation des jetons JWT.
- Retourner une erreur HTTP appropriée lorsque le jeton est invalide ou absent.

Exemples de routes publiques de l'application :

URL exposée	Service cible
/api/produits/**	ms-produits
/api/commandes/**	ms-commandes

La Gateway ne doit pas contenir les règles métier de gestion des produits ou des commandes.

5.4. Spring Cloud Config Server — configuration centralisée

```
classDiagram
    class ConfigServerApplication {
        +main(args)
    }
    class ConfigServerConfig {
        +activerConfigServer()
    }
    class ConfigurationRepository {
        <<Git ou stockage local>>
        +charger(application, profil)
    }
```

```
    class ConfigClient {
        +chargerConfiguration()
    }
    class ProduitApplication {
        <<client Config>>
    }
    class CommandeApplication {
        <<client Config>>
    }
    ConfigServerApplication --> ConfigServerConfig
    ConfigServerConfig --> ConfigurationRepository
    ConfigClient --> ConfigurationRepository : requête HTTP
    ProduitApplication --> ConfigClient
    CommandeApplication --> ConfigClient
```

Travail demandé :

- Créer config-server.
- Activer @EnableConfigServer.
- Stocker les fichiers de configuration dans un dépôt Git.
- Prévoir une configuration spécifique pour chaque application et chaque environnement.
- Faire charger les paramètres de connexion à la base, les ports et les paramètres Eureka par les clients Config.

Exemple de fichiers de configuration :

```
config-repo/
├── ms-produits.yml
├── ms-commandes.yml
├── api-gateway.yml
├── eureka-server.yml
└── application.yml
```

Les mots de passe et secrets ne doivent pas être stockés en clair dans un dépôt Git public.

5.5. Keycloak — authentification et autorisation

Keycloak est un serveur d'identité. Son diagramme est donc conceptuel : il ne s'agit pas de redéfinir ses classes internes, mais de montrer les principaux objets configurés dans le cadre du TP.

```
classDiagram
    class Realm {
        +String nom
    }
    class Client {
        +String clientId
        +String protocol
        +String accessType
    }
    class Utilisateur {
        +String username
        +String email
    }
    class Role {
        +String nom
    }
    class JetonJWT {
        +String issuer
        +String subject
        +Date expiration
        +List roles
    }
    Realm "1" o-- "*" Client
```

```
Realm "1" o-- "*" Utilisateur
Realm "1" o-- "*" Role
Utilisateur "*" -- "*" Role : possède
Client ..> JetonJWT : reçoit
```

Travail demandé :

18. Démarrer Keycloak.
19. Créer un realm nommé gestion-commerciale.
20. Créer les rôles CLIENT, GESTIONNAIRE et ADMIN.
21. Créer un client pour l'application et configurer le mécanisme d'authentification adapté.
22. Créer des utilisateurs de test et leur attribuer des rôles.
23. Configurer la Gateway pour valider les JWT émis par Keycloak.
24. Configurer également les microservices métiers comme serveurs de ressources OAuth 2.0, afin qu'ils puissent vérifier les jetons lorsqu'ils sont appelés directement.
25. Restreindre les opérations sensibles selon les rôles.

Exemple de politique d'autorisation :

Opération	Rôle requis
Consulter les produits	CLIENT ou GESTIONNAIRE
Créer une commande	CLIENT
Modifier un produit	GESTIONNAIRE ou ADMIN
Supprimer un produit	ADMIN
Consulter toutes les commandes	GESTIONNAIRE ou ADMIN

Pour la sécurité, la Gateway est le point d'entrée normal, mais les microservices ne doivent pas faire confiance aveuglément à un appel qui contournerait la Gateway.

6. Ports et démarrage des applications

Voici une proposition de ports pour le développement local.

Application	Port
Keycloak	8080
Config Server	8888
Eureka Server	8761
API Gateway	8081
ms-produits	8082
ms-commandes	8083
PostgreSQL produits	5433
PostgreSQL commandes	5434

Les ports des bases de données correspondent à des ports hôtes proposés pour le TP ; ils peuvent être mappés vers le port interne PostgreSQL 5432.

Ordre conseillé de démarrage

26. Keycloak et les bases de données.
27. Config Server.
28. Eureka Server.
29. ms-produits et ms-commandes.
30. API Gateway.
31. Tests avec Postman ou curl.

Pour les versions, il faut utiliser un couple compatible de Spring Boot et Spring Cloud. À la date du TP, la documentation officielle présente Spring Cloud 2025.1.x pour Spring Boot 4.0.x et Spring Cloud 2025.0.x pour Spring Boot 3.5.x. Il est préférable de générer tous les projets avec le même couple de versions compatible, plutôt que de mélanger arbitrairement les versions.

7. Déroulement du TP

Étape 1 — Création des projets

Durée indicative : 1 h

Créer les six projets Spring Boot nécessaires : les deux microservices métiers, Eureka, Config Server et API Gateway, puis préparer Keycloak. Ajouter les dépendances appropriées.

Étape 2 — Développement des services métiers

Durée indicative : 2 h

Implémenter les entités, repositories, services et contrôleurs REST de ms-produits et ms-commandes. Tester chaque API séparément.

Étape 3 — Découverte et communication

Durée indicative : 2 h

Configurer Eureka, enregistrer les applications et implémenter l'appel de ms-produits depuis ms-commandes à l'aide d'OpenFeign.

Étape 4 — Configuration centralisée et Gateway

Durée indicative : 1 h 30

Configurer Config Server, externaliser les paramètres, déclarer les routes et vérifier que les API sont accessibles par la Gateway.

Étape 5 — Sécurisation et validation

Durée indicative : 2 h

Configurer Keycloak, protéger les API, appliquer les règles d'autorisation et tester les scénarios nominaux et les erreurs.

8. Scénarios de tests à réaliser

Les étudiants devront démontrer le fonctionnement de l'ensemble avec Postman.

Test	Résultat attendu
Ajouter un produit valide	201 Created
Consulter un produit existant	200 OK
Consulter un produit inexistant	404 Not Found
Créer une commande avec un produit valide	201 Created
Créer une commande avec un produit inexistant	Rejet explicite
Créer une commande avec une quantité négative	400 Bad Request
Appeler une route protégée sans jeton	401 Unauthorized
Modifier un produit sans rôle autorisé	403 Forbidden
Appeler une API via la Gateway	Routage vers le bon microservice
Arrêter temporairement ms-produits	Gestion contrôlée de l'indisponibilité

Les codes HTTP sont des résultats attendus à implémenter ; il appartient aux étudiants de configurer la gestion des exceptions pour les obtenir.

9. Travail complémentaire

- Ajouter une pagination et un tri dans la liste des produits.
- Ajouter une validation des données avec jakarta.validation.
- Documenter les API avec OpenAPI / Swagger.
- Ajouter des tests unitaires et des tests d'intégration.
- Mettre en place un mécanisme de circuit breaker avec Resilience4j.
- Conteneuriser les applications avec Docker Compose.
- Ajouter un mécanisme de réservation de stock et réfléchir à la cohérence des transactions entre services.

10. Livrables attendus

Chaque groupe devra remettre :

32. Le code source des applications.
33. Les diagrammes UML des microservices.
34. Les fichiers de configuration et, si utilisé, le fichier Docker Compose.
35. Une collection Postman contenant les scénarios de tests.
36. Des captures d'écran du tableau de bord Eureka et de la configuration Keycloak.
37. Un rapport expliquant l'architecture, les choix techniques, les difficultés rencontrées et les tests effectués.